

GOLANG

Living Textbook cho Software Engineering và DevOps/SRE

Edition nền móng

Được kiểm chứng với Go 1.27.1 - 20-09-2026

Markdown là nguồn gốc. PDF là bản đọc được, có thể tái tạo cục bộ.

Mục lục của edition này

Đây là edition nền móng. Mục lục tổng thể và thứ tự các phần tiếp theo được giữ trong `book/README.md` để có thể mở rộng mà không giả vờ rằng chúng đã được viết xong.

- Mở cửa vào Go
- Chương 1 — Đọc và viết một chương trình Go

Mở cửa vào Go

Bạn không cần thuộc ngay danh sách keyword để bắt đầu. Điều cần trước tiên là nhìn một tệp Go và biết: phần nào đặt tên cho chương trình, phần nào tạo giá trị, phần nào quyết định đường đi, và phần nào chỉ là một lời gọi hàm.

Cuốn sách này không đi bằng các mẫu bài học quá ngắn. Nó đi bằng những lần thay đổi cách nhìn: đọc được code trước, nhìn được giá trị chạy qua code sau, rồi mới đến những lúc nhiều goroutine, mạng, dữ liệu và production khiến trực giác ban đầu không còn đủ. Khi một chủ đề cần một lỗi, một trace hay một thí nghiệm để thấy rõ, chương đó sẽ bắt đầu từ bằng chứng ấy.

Cách đọc: đừng lướt qua code block. Mỗi lần gặp một đoạn ngắn, hãy thử nói thành lời nó tạo giá trị gì, giá trị đó được đặt tên ở đâu và điều gì sẽ xảy ra nếu đổi một token. Compiler là phản hồi nhanh nhất cho dự đoán của bạn.

Chương đầu không có mục tiêu “biết hết Go”. Mục tiêu nhỏ hơn, nhưng thiết yếu: bạn có thể đọc một chương trình console ngắn từ trên xuống, tự sửa nó, và hiểu vì sao một thay đổi là hợp lệ hoặc không hợp lệ.

Chương 1 — Đọc và viết một chương trình Go

Một tệp nguồn là một lời khẳng định có cấu trúc, không phải một chuỗi keyword. Ta sẽ bắt đầu từ một chương trình rất nhỏ: nó nhận một mã trạng thái và in ra một nhãn mà người trực vận hành có thể đọc. Cố ý chọn ví dụ này vì nó đủ thực để có dữ liệu, lựa chọn và thông điệp, nhưng chưa che mất ngôn ngữ dưới một dự án.

```
package main

import "fmt"

const service = "checkout"

func classify(status int) string {
    if status >= 500 {
        return "failed"
    }
    return "healthy"
}

func main() {
    status := 503
    label := classify(status)
    fmt.Println(service, label)
}
```

Chưa cần đọc từng dòng. Hãy chỉ theo ba câu hỏi: chương trình bắt đầu ở đâu, giá trị nào được tạo, và giá trị nào bị gửi ra màn hình? Câu trả lời là main, literal 503 cùng các giá trị trả về từ classify, và lời gọi Println.

Bản đồ của một tệp nguồn

Từ đầu xuống cuối, tệp trên có bốn vùng: package nói tệp thuộc chương trình nào; import cho phép dùng tên từ package khác; khai báo ở cấp tệp tạo const và hàm; cuối cùng, thân hàm main là nơi một chuỗi câu lệnh được chạy.

Dòng hoặc mảnh code	Nó là gì khi đọc	Nó có tác dụng gì
package main	package declaration	Đặt tệp vào package tạo executable.
import "fmt"	import declaration	Cho phép tham chiếu fmt từ standard library.
const service = "checkout"	declaration + literal	Gắn một tên không đổi với chuỗi.
func classify(status int) string	function declaration + signature	Đặt tên hàm, tham số kiểu int và kết quả kiểu string.
status >= 500	expression	Tính ra một giá trị bool.
return "failed"	statement	Kết thúc lời gọi hiện tại với một string.
status := 503	short variable declaration	Tạo biến status trong block main.
fmt.Println(...)	function call statement	Gọi hàm để có hiệu ứng in ra stdout.

Các nhãn này không phải để học thuộc. Chúng cho ta một cách chỉ vị trí khi compiler báo lỗi: lỗi có thể nằm trong một biểu thức, kiểu của một lời gọi, hay phạm vi của một tên — không phải mơ hồ là “code bị sai”.



Hình 1 — Đây là mô hình đọc, không phải sơ đồ nội bộ chính xác của Go compiler. Nó giải thích vì sao một dấu ngoặc, một tên không tồn tại và phép cộng sai kiểu thường bị bắt trước khi chương trình có cơ hội chạy.

Một tên chỉ có nghĩa trong phạm vi của nó

Ở dòng `const service`, `service` là identifier: một cái tên do người viết chọn. `fmt`, `Println`, `classify`, `status` và `label` cũng là identifier. Ngược lại, `package`, `import`, `const`, `func`, `if` và `return` là keyword; chúng dành riêng cho cấu trúc ngôn ngữ nên bạn không thể dùng chúng để tự đặt tên.

Tên không tự mang giá trị. Câu `status := 503` tạo một biến, rồi gán giá trị literal 503 vào biến đó. Trong cùng block, câu `label := classify(status)` tạo một biến khác, nhận kết quả mà hàm trả về. Tên tồn tại từ điểm khai báo đến cuối block bao quanh.

```
func main() {
    status := 503
    if status >= 500 {
        label := "failed"
        fmt.Println(label)
    }
    // fmt.Println(label) // không hợp lệ: label đã hết scope
}
```

Khối `if` được tạo bởi cặp ngoặc nhọn. `label` chỉ hữu ích bên trong khối đó. Đây là một giới hạn có chủ ý: người đọc không phải lục lại toàn bộ tệp để đoán một tên có còn sống hay không.

Dừng lại để dự đoán: nếu bỏ comment ở dòng cuối, compiler phàn nàn về giá trị hay về tên? Câu trả lời là tên: `label` không còn được khai báo trong scope mà lời gọi `Println` đang đứng.

Làm chương trình lớn lên, mỗi lần một ý

Đừng viết đầy đủ từ đầu. Ta quan sát cùng một ý định khi nó nhận thêm một yêu cầu. Mỗi phiên bản giữ lại phần cũ có ích và chỉ thêm một khái niệm.

Literal: một kết quả cố định

```
package main

import "fmt"

func main() {
    fmt.Println("checkout healthy")
}
```

"checkout healthy" là string literal. Chương trình có một statement gọi hàm và không có quyết định nào. Nó đúng nếu thông điệp không bao giờ thay đổi.

Variable: dữ liệu thay đổi được đặt tên

```
func main() {
    service := "checkout"
    label := "healthy"
    fmt.Println(service, label)
}
```

Short declaration `:=` vừa tạo tên vừa gán giá trị. Nó chỉ hợp lệ bên trong thân hàm. Nếu `service` đã tồn tại trong cùng scope, dùng `=` khi chỉ muốn thay giá trị:

```
label = "degraded"
```

Muốn làm gì?	Viết gì?	Điều compiler cần biết
Tạo một biến mới và suy ra kiểu	<code>name := value</code>	Ít nhất một tên ở vế trái là mới trong scope hiện tại.
Gán lại biến đã có	<code>name = value</code>	Tên đã tồn tại và giá trị mới phù hợp kiểu.
Đặt giá trị không đổi ở cấp tệp	<code>const name = value</code>	Giá trị phải là hằng; không dùng <code>:=</code> .
Khai báo rõ kiểu trước	<code>var name Type = value</code>	Dùng khi kiểu hoặc zero value cần được nói rõ.

Expression: để chương trình tự kết luận

Literal 503 không tự nói rằng service thất bại. Ta cần một biểu thức tạo ra quyết định. `status >= 500` đọc là “status lớn hơn hoặc bằng 500”, và kết quả của nó là bool: true hoặc false.

```
func main() {  
    status := 503  
    if status >= 500 {  
        fmt.Println("checkout failed")  
    }  
}
```

if nhận một biểu thức bool. Khi biểu thức là true, block của if được thực hiện; khi false, nó bị bỏ qua. Dấu ngoặc không chỉ để trang trí: chúng xác định chính xác nhóm câu lệnh phụ thuộc vào điều kiện.

Toán tử có thứ tự ưu tiên. Viết ngoặc khi chúng làm ý định rõ hơn thay vì bắt người đọc nhớ luật ưu tiên:

```
urgent := status >= 500 ||  
    (status >= 400 && service == "checkout")
```

`&&` nghĩa là cả hai vế phải đúng; `||` nghĩa là ít nhất một vế đúng. Biểu thức bên trong ngoặc chạy trước phần `||`. Không cần biến mọi phép toán thành một câu đố — một tên như `urgent` thường dễ đọc hơn một điều kiện bị nén.

Function: đặt tên cho một quyết định có thể tái dùng

Khi điều kiện có ý nghĩa riêng, đưa nó vào hàm. Đây không phải “cho code đẹp”; nó cho người đọc một tên ở đúng mức trừu tượng.

```
func classify(status int) string {  
    if status >= 500 {  
        return "failed"  
    }  
    return "healthy"  
}
```

Signature nói rõ hợp đồng tối thiểu: `classify` nhận một `int` và trả về một `string`. `status` là parameter, chỉ tồn tại trong lời gọi hàm. `return` không in gì cả; nó gửi một giá trị về nơi gọi. Vì thế câu `label := classify(status)` đọc theo thứ tự này: đọc `status` hiện có, gọi `classify` với giá trị đó, nhận `string` kết quả, rồi tạo `label`.

Cùng một quyết định, hai cách tổ chức

Nếu có nhiều trạng thái rời rạc, `switch` thường bộc lộ ý định tốt hơn một chuỗi `if`. Nó không “mạnh hơn”; nó chỉ đặt các nhánh cùng cấp cạnh nhau.

Đầu vào	Dạng phù hợp	Lý do đọc dễ hơn
Một ngưỡng: <code>status >= 500</code>	<code>if</code>	Điều kiện là một mệnh đề đúng/sai.
Vài giá trị cụ thể: 200, 404, 503	<code>switch</code>	Mỗi case là một lựa chọn cùng cấp.
Một thao tác lặp trên danh sách	<code>for</code> hoặc <code>range</code>	Ý định là làm lại, không phải chọn một nhánh.

```
func severity(status int) string {
    switch {
    case status >= 500:
        return "critical"
    case status >= 400:
        return "warning"
    default:
        return "normal"
    }
}
```

`switch` trên không có expression sau từ `switch`, nên từng case là bool expression. Go chạy case đầu tiên đúng rồi rời `switch`; không có `fallthrough` ngầm như một số ngôn ngữ khác. Thứ tự ở đây là dữ liệu của thiết kế: nếu đưa `status >= 400` lên trước, 503 sẽ bị phân loại `warning`. Một compiler không thể bắt lỗi ý nghĩa đó cho bạn; đó là việc của người đọc và test.

Lặp là đi qua một tập giá trị

Một for có thể diễn tả vòng lặp điều kiện, nhưng range thường trực tiếp hơn khi ta muốn đi qua các phần tử của một collection.

```
statuses := []int{200, 503, 404}

for _, status := range statuses {
    fmt.Println(status, severity(status))
}
```

[]int là slice của int; cặp ngoặc nhọn tạo literal gồm ba phần tử. range cung cấp index và value. Tên _ bảo Go bỏ index đi vì ta không cần nó. Mỗi lượt, status là giá trị hiện tại; lời gọi severity dùng nó và Println in kết quả. Ta sẽ quay lại slice, bộ nhớ và aliasing ở chương sau — hiện tại chỉ cần thấy luồng đọc của vòng lặp.

Kiểu không phải nghi thức: nó giới hạn những phép có nghĩa

Go suy ra nhiều kiểu từ literal, nhưng vẫn kiểm tra chúng chặt. int biểu diễn số nguyên, string biểu diễn chuỗi byte được hiểu theo UTF-8, bool biểu diễn true/false. Một biến có kiểu xác định; phép toán đòi các vế tương thích.

```
status := 503
message := "status: " + status // không hợp lệ
```

Toán tử + ở đây có thể nối hai string hoặc cộng hai số, nhưng không tự đoán cách trộn string và int. Chuyển đổi là một quyết định phải viết ra:

```
message := "status: " + fmt.Sprint(status)
```

fmt.Sprint tạo string để dùng cho hiển thị. Trong những ranh giới nghiêm túc hơn, ta sẽ phân biệt conversion với parsing và kiểm tra lỗi. Bây giờ điểm quan trọng là: kiểu giúp phát hiện một phép ghép vô nghĩa ngay tại nơi bạn viết nó.

Đọc compiler như một người cộng tác khó tính

Hãy tạo thư mục lab part1-reading-go rồi chạy tệp main.go. Thử từng thay đổi một, hoàn tác sau mỗi lần, và đọc lỗi trước khi sửa.

```
go run .  
go test ./...  
go vet ./...
```

Thay đổi thứ nhất: đổi `status := 503` thành `status = 503`. Khi chưa có `status` trong scope, compiler sẽ nói tên chưa được khai báo. Thay đổi thứ hai: đổi tham số của `classify` từ `int` thành `string` nhưng vẫn gọi với `status`. Lần này lỗi nói về kiểu của đối số. Hai lỗi đều là phản hồi cấu trúc: compiler không đoán hộ bạn bạn muốn tạo tên mới hay chuyển một số thành chuỗi.

Bài đọc cuối chương: không cần viết chương trình mới. Hãy nhìn hàm `severity`, dự đoán kết quả cho 200, 404 và 503, rồi viết một test table nhỏ cho ba dự đoán đó. Nếu test thất bại, lần theo branch được chọn thay vì sửa output một cách mù.

Ở đây ta đã có từ vựng để đọc phần lớn tệp Go nhỏ: package đưa tệp vào một đơn vị, khai báo đặt tên, biểu thức tạo giá trị, statement điều phối hiệu ứng, block giới hạn scope, và hàm biến một đoạn quyết định thành một đơn vị có hợp đồng. Chương tiếp theo sẽ không lặp lại danh sách này. Nó sẽ hỏi câu khó hơn: khi giá trị đi qua hàm, copy, slice và string, dữ liệu nào thực sự được chia sẻ?